

CÉU DE BRIGADEIRO - AVALIAÇÃO DO USO DA HEURÍSTICA NA BIBLIOTECA CFGRIB

2023005594 - Gabriel Carneiro Roque Toti Silva





Céu de Brigadeiro - Avaliação do uso da heurística na biblioteca CFGRIB

1 Introdução

Atualmente, no pipeline do projeto Céu de Brigadeiro são utilizadas duas bibliotecas python para a leitura de arquivos *.grib*: o *xarray* — Uma biblioteca inspirada no pandas e no formato de arquivo netCDF, construída para lidar com dados de múltiplas camadas — e o *cfgrib* — construído em cima da biblioteca *ecCodes* do ECMWF —, sendo que a primeira fornece a estrutura de dados em memória utilizada durante o processamento das informações, enquanto a segunda é responsável por ser o motor que faz a conversão das informações presentes nos arquivos *.grib* formatadas em mensagens para as estruturas de dados do *xarray*. A conversão mencionada é parte fundamental do processo e permite que os dados sejam efetivamente lidos em um formato que suporta múltiplas camadas, podendo ser feita através de duas funções: a *open_dataset* e a *open_datasets*. Entretanto, como explicitado em ECMWF (2024), Arquivos Grib Heterogêneos podem causar conflitos na leitura em dois casos. O primeiro deles ocorre na impossibilidade de gerar um único *hypercube* devido a uma variável que possui mais de um *typeOfLevel*, como *isobaricInhPa* e *hybridTypeOfLevel*. O outro se dá quando uma chave do tipo *coord* (coordenada) possui valores distintos no mesmo arquivo, mesmo que para variáveis distintas, como por exemplo, a *heightAboveGround*, que, no contexto do *pipeline* atual, pode possuir os valores 2, 10 e 100 a depender da variável observada.

Para contornar este problema, ECMWF (2024) sugere utilizar filtros na função *open_dataset*, como por exemplo

```
1 cfgrib.open_dataset(  
2     'nam.t00z.awp21100.tm00.grib2',  
3     engine='cfgrib',  
4     backend_kwargs={'filter_by_keys':  
5         {'typeOfLevel': 'isobaricInhPa'}}  
6 )
```

que permite a extração das variáveis que apresentam apenas o *typeOfLevel* igual a *isobaricInhPa*, ou então utilizar a própria função *open_datasets* pura, que utiliza uma heurística para abstrair parte do processo de filtragem e pode simplificar a extração dos dados, especialmente no caso de uso de fontes diversas em vários processamentos, como neste projeto. Com isso em mente, o presente documento possui como objetivo entender as razões pelas quais os conflitos citados ocorrem, descrever o funcionamento da heurística utilizada pelo *open_datasets* e avaliar o desempenho das opções de leitura disponíveis.

2 Referencial Teórico

Buscando compreender o funcionamento da heurística, o código fonte da biblioteca *cfgrib* disponível em ECMWF (2024) foi analisado por meio da ferramenta de *debug* do *Visual Studio Code* a partir da implementação do *pipeline* realizada até o momento de escrita deste documento, isto é, os serviços *clip_variable_service* e *process_wind_service*. O funcionamento da heurística será descrito no desenvolvimento.

2.1 Estruturas de dados

Além disso, a documentação da biblioteca *xarray* foi estudada com o foco em entender as principais estruturas de dados utilizadas durante o processamento, baseando-se principalmente em for Data (2014b) para o *DataArray*, for Data (2014a) para o *Dataset*, for Data (2014d) para a *Variable* e for Data (2014c) para uma visão geral e exemplos de uso, juntamente com uma apresentação da biblioteca (Hoyer 2015) feita durante o evento *ScyPy 2015*.

2.1.1 *DataArray*

O *DataArray* é uma estrutura de dados que encapsula um array multidimensional do *numpy* com labels para indexação de dimensões e coordenadas, juntamente com outros atributos relevantes para os dados armazenados, permitindo processamentos "*metadata aware*". Dentre as propriedades contidas nessa estrutura de dados estão:

- *values*: Um array multidimensional do *numpy* que armazena os dados propriamente ditos;
- *dims*: Uma lista com os nomes dados para cada dimensão do array, como latitude, longitude, tempo, x, y, etc;
- *coords*: Um dicionário de dados utilizado para indexar e agregar informações aos valores do array. Os valores de cada uma dessas coordenadas pode ser do tipo dimensional ou não-dimensional. Por exemplo: a latitude e a longitude podem ser duas das dimensões dos dados armazenados no array, entretanto, seus valores não são armazenados diretamente nele, nesse caso, ambos os valores podem ser armazenados no dicionário de coordenadas como um array unidimensional, sendo coordenadas dimensionais, isto é, estão diretamente relacionadas com as dimensões do array e são utilizadas para sua indexação, marcados por um *''** quando são printados no console. Em oposição a isso, estão as coordenadas não-dimensionais, que não possuem o mesmo nome das dimensões e nem relação direta com as mesmas, não sendo, portanto, utilizadas para indexar os valores;

- *attrs*: Um dicionário para armazenar metadados arbitrários. Esses dados não possuem nenhum papel nas funções nativas do Xarray, o que não impede que sejam utilizados pelo programador para algum fim específico de sua aplicação (for Data, 2014c).

2.1.2 Dataset

O *Dataset* é uma estrutura de Dicionário equivalente ao *DataFrame* do pandas para dados multidimensionais (for Data, 2014c), agrupando diversas variáveis em uma única estrutura. Ela encapsula e armazena um *DataArray* (na prática o tipo retornado é uma *Variable*) para cada variável, sendo que todos os arrays tem as mesmas dimensões, coordenadas e atributos. Os campos desse tipo de dado estão descritos abaixo:

- *dims*: Um Dicionário onde as chaves são nomes das dimensões dos arrays e os valores são os comprimentos de cada dimensão (Isso está previsto para ser alterado para o mesmo padrão do *DataArray*);
- *data_vars*: Um dicionário de arrays que armazena os *DataArrays* das variáveis;
- *coords*: Um dicionário de dados que armazena *DataArrays* que servem como labels para os valores no *data_vars*. Note que a única diferença entre o *coords* e o *data_vars* é semântica: os valores do primeiro são pra valores constantes/independentes, enquanto o segundo é para valores variáveis. Essa diferença será levada em consideração quando os cálculos forem executados por funções nativas do *xarray* (for Data, 2014c);
- *attrs*: Um dicionário para armazenar metadados arbitrários.

Uma representação da estrutura de um *Dataset* pode ser vista na imagem abaixo:

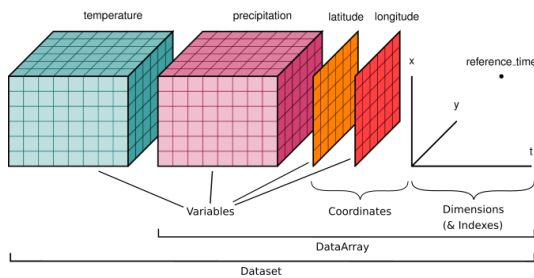


Figura 1: for Data (2014c)

Ao observar como as *coords* são utilizadas em ambas as estruturas de dados, é possível deduzir as razões pelas quais os conflitos mencionados podem ocorrer, isto é, a indexação e o uso de chaves para identificar os valores no dicionário impedem os valores de chave repetidos e o uso de múltiplos valores para indexação em uma mesma dimensão.

2.2 Arquivos grib

Por fim, outra estrutura avaliada, que será importante para compreender as diferenças de performance obtidas durante os

testes, é a estrutura de mensagens do arquivo grib, que foi brevemente avaliada sob a ótica do ECMWF *ecCodes* em ECMWF (2025).

Os dados do ECMWF recebidos pelo pipeline estão gravados no formato de arquivo .grib (General Regularly-distributed Information in Binary form), que é um formato compacto feito para lidar com grandes quantidades de dados. Esse formato é definido por uma ou mais mensagens — que podem estar em mais de uma versão grib em um mesmo arquivo — divididas em diferentes seções agrupando octetos, cada uma armazenando um tipo de informação sobre os dados, sendo que a leitura de cada uma das mensagens é feita de maneira sequencial. Abaixo está uma figura que representa a estrutura de um arquivo grib de maneira simplificada:

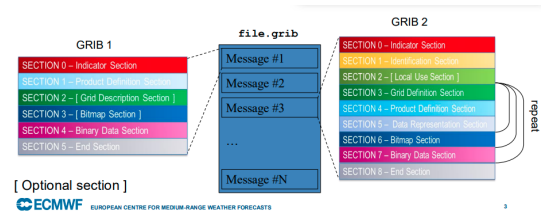


Figura 2: ECMWF (2025)

2.3 Arquivos de índice

Como dito anteriormente, a leitura pura de um arquivo grib é feita de maneira sequencial, mensagem por mensagem. Esse processo é lento e computacionalmente custoso, exigindo uma leitura $O(n)$ para cada vez que o arquivo precisa ser aberto e, como será visto posteriormente, isso possui um grande impacto na primeira etapa da heurística.

Para solucionar este problema, o *Cfgrib* permite o uso de arquivos de índice que armazenam algumas informações sobre os dados de cada arquivo grib, sendo a posição do primeiro byte de cada mensagem a mais relevante para o contexto deste documento.

3 Desenvolvimento

Nesta seção serão descritos os testes realizados nas funções, bem como o ambiente e a máquina utilizados para este propósito. Contudo, antes de adentrar nestes detalhes, é necessário compreender o funcionamento da heurística utilizada pela função *open_datasets* e o papel dos arquivos de índice, que podem ou não ser aplicados, na performance da função.

3.1 Heurística

A heurística descrita a seguir permite solucionar os problemas de múltiplos *hypercubes* para a mesma variável e múltiplos valores para uma mesma coordenada, citados durante a introdução deste documento, e pode ser dividida em 3 etapas principais após a leitura do arquivo, onde se obtém alguns metadados do arquivo, como *paramId*.

O primeiro passo é iterar sobre cada um dos *paramIds* obtidos e utilizar a função padrão de leitura de *Datasets* do *xarray*, filtrando as variáveis pelo *paramId*. Isso faz com que cada uma das variáveis seja carregada em um *Dataset* separado na memória. Nesta etapa, a função de leitura do *xarray*

pode identificar um parâmetro com múltiplos *hypercubes*, lançando uma excessão, que é capturada pelo *cfgrib*, fazendo com que a função de separação seja chamada de forma recursiva, uma vez para cada parâmetro. Aqui, como cada um dos passos do loop significa uma nova abertura do arquivo grib que está sendo processado, o arquivo de índice é essencial para que não seja necessário ler todo o *grib* várias vezes:

```

1 def raw_open_datasets(path, backend_kwargs
2     = {}, **kwargs):
3     # type: (str, T.Dict[str, T.Any], T.Any)
4     #       -> T.List[xr.Dataset]
5
6     fbks = []
7     datasets = []
8
9     try:
10        # Calls xarray default open_dataset
11        # with cfgrib as the engine
12        datasets.append(open_dataset(path,
13            backend_kwargs=backend_kwargs,
14            **kwargs))
15    except DatasetBuildError as ex:
16        fbks.extend(ex.args[2])
17
18    # NOTE: the recursive call needs to
19    # stay out of the exception handler
20    # to avoid showing
21    # to the user a confusing error message
22    # due to exception chaining
23
24    for fbk in fbks:
25        bks = backend_kwargs.copy()
26        bks["filter_by_keys"] = fbk
27        datasets.extend(raw_open_datasets(
28            path, backend_kwargs=bks, **
29            kwargs))
30
31    return datasets

```

Com cada uma das variáveis em um *Dataset*, a lista de datasets é iterada, o *GRIB_typeOfLevel* de cada *Dataset* é lido e utilizado para agrupar os *Datasets* dentro de um dicionário, cujas chaves são os *GRIB_typeOfLevels* e os valores são os *Datasets*. Isso ocorre ignorando conflitos citados em relação a tipos de níveis iguais com valores diferentes, como *heightAboveGround* com múltiplos valores:

```

1 type_of_level_datasets = {} # type: T.Dict[
2     str, T.List[xr.Dataset]]
3
4 for ds in datasets:
5     for _, da in ds.data_vars.items():
6         type_of_level = da.attrs.get(
7             "GRIB_typeOfLevel", "undef")
8         type_of_level_datasets.setdefault(
9             type_of_level, []).append(ds)

```

A última etapa envolve validar a compatibilidade entre os *Datasets* agrupados e utilizar a função *xarray.merge* para unir todas as variáveis compatíveis entre si em um único *Dataset*. Para isso a lista de *Datasets* de cada tipo de nível é iterada, o merge é aplicado e os datasets resultantes são adicionados a lista que será retornada como resultado do processo todo:

```

1 merged = [] # type: T.List[xr.Dataset]
2 for type_of_level in sorted(
3     type_of_level_datasets):

```

```

3     for ds in merge_datasets(
4         type_of_level_datasets[type_of_level],
5         join="exact", combine_attrs="
6         identical"
7     ):
8         merged.append(ds.squeeze() if
9             squeeze else ds)

```

A validação mencionada é feita dentro da função *merge_datasets*. Nela são utilizadas duas listas para controlar os *Datasets* gerados: *merged*, para armazenar todos os resultados e *first*, para armazenar apenas os primeiros *Datasets* identificados para determinado grupo. Dentro do loop que itera pelo conjunto de *Datasets*, há um segundo loop que itera pela lista *merged* e verifica se o *Dataset* do loop mais externo possui todos os atributos presentes no *Dataset* de *merged* compatíveis e em caso positivo os *Datasets* são unificados e o loop mais interno é interrompido, caso contrário, o *Dataset* mais externo é adicionado às listas *merged* e *first*. Se quando a função *xr.merge* for executada houver um erro, o loop não atinge o *break* e o *Dataset* será considerado um novo tipo de *Dataset* e adicionado às listas *merged* e *first*, resolvendo o problema em aberto dos tipos de nível com valores incompatíveis ignorado durante o agrupamento:

```

1 # type: (T.Sequence[xr.Dataset], T.Any) ->
2 T.List[xr.Dataset]
3
4 merged = [] # type: T.List[xr.Dataset]
5 first = [] # type: T.List[xr.Dataset]
6
7 for ds in datasets:
8     ds.attrs.pop("history", None)
9     for i, o in enumerate(merged):
10        if all(
11            o.attrs[k] == ds.attrs[k]
12            for k in o.attrs
13        ):
14            try:
15                o = xr.merge([o, ds], **kwargs)
16                o.attrs.update(ds.attrs)
17                merged[i] = o
18                break
19            except Exception:
20                pass
21        else:
22            merged.append(ds)
23            first.append(ds)

```

Após todas estas etapas, a lista *merged* pode ser retornada para o usuário.

3.2 Testes

Os testes de performance foram realizados considerando 3 possibilidades de leitura de arquivos *grib*: Leitura padrão, que utiliza apenas o *open_dataset*, nesse caso, filtrando por tipos de níveis específicos para evitar os conflitos; a Heurística, que utiliza o *open_datasets* sem os arquivos de índice; e a Heurística Indexada, que faz uso do *open_datasets* com os arquivos de índice.

Para a execução dos testes de cada um dos casos, foi gerado um script python simples que realiza a leitura de todos os steps disponíveis dos arquivos abertos do ECMWF do dia

28 de janeiro de 2026 contendo apenas as variáveis `u`, `v`, `tp`, `sp` e `2t`. O script foi executado através do Apache JMeter, que foi configurado para executá-lo 30 vezes para cada caso de teste, levando em consideração que no caso em que o arquivo de índice é utilizado, um pré-processador precisa remover todos os arquivos de índice utilizados pela execução anterior. As condições de leitura estabelecidas são análogas aos casos aos quais o pipeline do projeto pode ser exposto, entretanto, é necessário reconhecer que o escopo das variáveis está bastante reduzido, o que resulta em tempos médios significativamente menores do que um cenário real poderia apresentar. O script pode ser observado abaixo:

```

1  import cfgrib
2  import os
3  import sys
4
5  reading_types = ('pure_heuristic', '
    indexed_heuristic', 'default')
6  reading_option = sys.argv[1]
7
8  if reading_option not in reading_types:
9      print(f"Tipo invalido: {reading_option
    }")
10     print(f"Escolha um dos seguintes: {str(
    reading_types)[1:-1]}")
11     exit(1)
12
13 print(f'Lendo com: {reading_option}')
14
15 data_path = 'data'
16 files = os.listdir(data_path)
17
18 def read_file(file: str, reading_option:
    str):
19     if reading_option == reading_types[0]:
20         g_data = cfgrib.open_datasets(file,
    backend_kwargs={'indexpath
    ': ''})
21
22         for ds in g_data:
23             if ds.get('isobaricInhPa') is
    not None:
24                 break
25     elif reading_option == reading_types
    [1]:
26         index_path = os.path.join(data_path
    , "index_files")
27         os.makedirs(index_path, exist_ok=
    True)
28         file_name = os.path.basename(file)
29         index_file = os.path.join(
    index_path, file_name + ".idx")
30
31         g_data = cfgrib.open_datasets(file,
    backend_kwargs={'indexpath':
    index_file})
32
33         for ds in g_data:
34             if ds.get('isobaricInhPa') is
    not None:
35                 break
36     else:
37         cfgrib.open_dataset(file,
    backend_kwargs={'filter_by_keys
    ': {'typeOfLevel': '

```

```

isobaricInhPa'}, 'indexpath':
'')

```

```

38
39 for file in files:
40     file_path = os.path.join(data_path,
    file)
41     if not os.path.isfile(file_path):
42         continue
43     read_file(file_path, reading_option)
44 print("Leitura concluida")

```

Como forma de avaliar a performance de cada caso de teste, foram escolhidos os seguintes parâmetros: uso de CPU, considerando que a cada 100% de uso significa o uso de um core completo, ou seja, 150% significa 100% de um core e 50% de outro; uso de Memória RAM em MB; Tempo de execução em segundos; e uso adicional de espaço em disco, isto é, espaço utilizado além dos grib's baixados e do script já existente. Por fim, todos os dados foram coletados através da ferramenta *time* do linux — exceto pelo uso de disco que foi medido pelo *Disk Usage Analyzer* do GNOME — e extraídos por um Pós-processador do JMeter em uma máquina com as seguintes configurações:

- Processador: Intel(R) Core(TM) i5-4460 CPU @ 3.20GHz
- Memória RAM: 16GB DDR3 1600MHz
- Memória secundária: SSD Kingstom 1TB
- Sistema Operacional: ZorinOS 18

4 Resultados

Abaixo estão descritos os resultados médios de cada caso de teste de acordo com os parâmetros especificados anteriormente:

Tabela 1: Comparação entre o resultado médio de cada teste

Método	Tempo (s)	Disco (KB)	CPU (%)	Memória (MB)
Padrão	6,3	0	106,96	245,05
Heurística	26,15	0	101	245,13
Heurística Indexada	9,39	335,9	102,93	245,12

Como é possível perceber pelos dados apresentados na Tabela 1, o uso de Memória RAM e CPU possuem variações pouco significativas entre si e que teriam pouca influência na performance do pipeline em um caso real. Contudo, não se pode descartar a diferença entre os tempos obtidos por cada um dos casos de teste, sendo que a Heurística obteve um tempo quase 3 vezes maior que a Heurística Indexada e mais de 4 vezes maior que o método padrão. Esse resultado, quando confrontado com o resultado da Heurística Indexada, indica que isso ocorre devido a necessidade de múltiplas leituras de cada arquivo para fazer a separação das variáveis em *Datasets* diferentes, juntamente à natureza sequencial da leitura dos arquivos grib, que é contornada pelo uso de índices. Em cenários com uma carga maior de variáveis, os tempos de leitura da Heurística poderiam tornar o processamento de vários steps inviável.

Já, ao observar os dados obtidos pela Heurística Indexada com o Padrão, é possível notar que a diferença é bastante diminuta, tanto para o tempo, quanto para o uso do disco, permitindo que a decisão entre o uso de um método ou outro possa ficar com a facilidade de uso, que, como já mencionado na introdução do documento, é maior com a heurística para casos em que se trabalha com mais de uma fonte e tipo de arquivo grib, como é o caso do projeto Céu de Brigadeiro (arquivos abertos e arquivos restritos fornecidos à NexAtlas).

5 Conclusões

A análise experimental demonstrou que o consumo de recursos computacionais (CPU e Memória RAM) mantém-se estável independentemente do método de leitura utilizado, isolando o tempo de execução como o principal critério de decisão. Os testes evidenciaram a ineficiência da abordagem heurística pura, cujo tempo de processamento se mostrou inviável para operações em larga escala, sendo mais de 4 vezes superior ao método padrão.

Entretanto, a introdução de arquivos de índice na abordagem heurística (*Heurística Indexada*) mitigou drasticamente a latência de leitura, aproximando seu desempenho do método padrão (9,39s contra 6,3s), com um custo de armazenamento em disco negligenciável. Considerando a arquitetura do projeto Céu de Brigadeiro, que necessita processar fontes de dados heterogêneas, conclui-se que a Heurística Indexada é a uma solução mais adequada. Ela oferece um bom desempenho enquanto permite uma simplificação da construção dos filtros, permitindo maior foco na extração das variáveis e níveis desejados sem a necessidade de se preocupar com o restante.

Referências

ECMWF (2024). Cfgrib. <https://github.com/ecmwf/cfgrib>.

ECMWF (2025). Grib decoding with eccodes: Introduction to ecmwf computing services. *Online training course: Introduction to ECMWF computing services (including MARS)*, 1.

for Data, P. (2014a). Data structures. <https://docs.xarray.dev/en/stable/user-guide/data-structures.html>.

for Data, P. (2014b). xarray.dataarray. <https://docs.xarray.dev/en/stable/generated/xarray.DataArray.html>xarray.DataArray.

for Data, P. (2014c). xarray.dataset. <https://docs.xarray.dev/en/stable/generated/xarray.Dataset.html>.

for Data, P. (2014d). xarray.variable. <https://docs.xarray.dev/en/stable/generated/xarray.Variable.html>xarray.Variable.

Hoyer, S. (2015). xray: N d labeled arrays and datasets. *ScyPy*, 2015.

